WIE

Ticketing System

Complete API Documentation for React Native Developers

Event Listing • Booking • Payments • Interactions • Cancellations

Event Listing: port 5005 | Booking & Interactions: port 5007 | v1.0

Table of Contents

PART A — Event Listing Service (Base URL: http://localhost:5005/api/tickets)

01 Service Overview & Authentication

The Event Listing Service runs on the WIE User Service at port 5005. It proxies event data from the internal Ticket Service via gRPC. All event discovery, search, filtering, and location-based queries go through these endpoints. No authentication is required for most listing endpoints.

TypeScript

```
Base URL (development) : http://localhost:5005/api/tickets
Base URL (Android emu) : http://10.0.2.2:5005/api/tickets
Most listing endpoints are PUBLIC (no token required).
saveUserLocation requires a userId body field.
// React Native Axios instance
const ticketApi = axios.create({
baseURL: Config.API_BASE_URL + '/api/tickets',
timeout: 20000,
});
```

02 Get All Live Events

Returns every currently live (active) event from the ticket service. Results include main events and their sub-events. Use this as the base data source for the home/discover screen.

Method	Endpoint	Auth	Description
GET	/live-events	—	Fetch all live events. No auth or query params needed.

Response

JSON Response

```
{
"success": true,
"message": "Live events fetched successfully",
"data": [
{
"_id": "ticket_abc",
"event_name": "WIE Music Fest 2025",
"event_category": "Music",
"event_banner": "https://cdn.../banner.jpg",
"event_portrait": "https://cdn.../portrait.jpg",
"location": "Mumbai, India",
"venue": "BKC Arena",
"payment_type": "paid",
"exact_map_location": { "latitude": 19.0596, "longitude": 72.8656, "address": "..." },
"event_dates": [ { "start_date": "2025-12-20", "start_time": "18:00" } ],
"ticket_types": [ { "_id": "tt_01", "ticket_type": "General", "ticket_price": 500 } ],
"totalBookings": 142,
"like": 38,
"sub_events": [ /* sub-event objects */ ]
}
]
```

```
}
```

03 Get All Events with Distance

Returns every live event enriched with a distance field (km) from the provided coordinates or location string. Events without coordinates get distance=null. Results are sorted nearest-first.

Method	Endpoint	Auth	Description
GET	/all-events	—	All live events with optional distance enrichment. Sorted nearest first.

Query Parameters

Field	Type	Req.	Description
latitude	number	No	GPS latitude. Pair with longitude.
longitude	number	No	GPS longitude. Pair with latitude.
location	string	No	Location name string (geocoded via Google Maps).

Examples

```
// With GPS
GET /all-events?latitude=19.0596&longitude;=72.8656
// With city name
GET /all-events?location=Mumbai
// No location — all events returned (distance: null)
GET /all-events
```

04 Get Nearby Events

Returns only events within a specified radius from the user's location. Supports both GPS coordinates and city name geocoding. Events with sub-events that fall inside the radius are included even if the main event is outside it.

Method	Endpoint	Auth	Description
GET	/nearby-events	—	Events within radius. Sorted nearest first. Sub-events checked independently.

Query Parameters

Field	Type	Req.	Description
latitude	number	No	GPS latitude. Required if longitude is provided.
longitude	number	No	GPS longitude. Required if latitude is provided.
location	string	No	City or address string (geocoded). Used if GPS not provided.
radius	number	No	Search radius in km. Min 1, Max 99999. Default: 30.

Response

JSON Response

```
{
```

```
"success": true,
"data": {
"count": 8,
"search_location": { "latitude": 19.05, "longitude": 72.86, "radius": 30, "radius_unit": "km" },
"events": [
{
"_id": "ticket_abc",
"event_name": "WIE Music Fest",
"distance": 4.2,
"distance_unit": "km",
"is_main_event_nearby": true,
"has_nearby_sub_events": false,
"nearby_sub_events": []
}
]
}
}
```


05 Initial Events (Home Screen Load)

The primary endpoint for the home screen. Automatically uses the user's saved profile location (latitude/longitude or location string) to filter events. Falls back to country-based filtering if no coordinates. Returns events grouped by category.

Method	Endpoint	Auth	Description
GET	/initial-events	—	Location-aware home screen events grouped by category. Pass userId for personalisation.

Query Parameters

Field	Type	Req.	Description
userId	string	No	User ID to fetch saved location and country from profile. Enables personalisation.

Response

JSON Response


```
{
"success": true,
"data": {
"locationAvailable": true,
"locationSource": "saved", // 'saved' | 'country' | 'none'
"searchLocation": { "latitude": 19.0, "longitude": 72.8 },
"searchRadius": 200,
"categories": ["Music", "Sports", "Comedy"],
"eventsByCategory": {
"Music": [ /* Event[] */ ],
"Sports": [ /* Event[] */ ]
},
"totalEvents": 24,
"countryCode": "IN",
"countryName": "India"
}
}
```

BEST PRACTICE: Call /initial-events on app launch passing the user's ID. The server resolves location automatically from the saved profile — no need to request GPS permissions upfront.

06 Search Events by Name

Free-text search across event name, description, and hashtags. Optionally filtered by the user's country. Returns results grouped by category.

Method	Endpoint	Auth	Description
GET	/search-by-name	—	Search events by keyword. Matches name, description, hashtags.

Query Parameters

Field	Type	Req.	Description
searchQuery	string	Yes	Search keyword. Minimum 1 character.
userId	string	No	User ID for optional country-level filtering of results.

Response

JSON Response

```
{
"success": true,
"message": "Found 5 event(s) matching 'music'",
"data": {
"searchQuery": "music",
"categories": ["Music"],
"eventsByCategory": { "Music": [ /* events */ ] },
"totalEvents": 5,
"countryCode": "IN",
"countryName": "India"
}
}
```

07 Search Events by Location

Location-specific event search. Supports GPS coordinates (radius-based) and city/address text (strict word-boundary matching). When no main results are found, returns suggestions from the user's profile location (up to 200 km).

Method	Endpoint	Auth	Description
GET	/search-by-location	—	Location-based search. GPS = radius filter. Text = strict address match.

Query Parameters

Field	Type	Req.	Description
latitude	number	No	GPS latitude. Pair with longitude.
longitude	number	No	GPS longitude. Pair with latitude.
location	string	No	City/area name (strict text matching against event addresses).
radius	number	No	Search radius in km (GPS mode only). Default: 50.

Field	Type	Req.	Description
userId	string	No	User ID for fallback suggestion location when no results found.

Response — includes suggestion fallback

JSON Response

```
{
"data": {
"locationAvailable": true,
"locationSource": "gps", // 'gps' | 'manual' | 'none'
"searchRadius": 50,
"categories": ["Music"],
"eventsByCategory": { "Music": [] },
"totalEvents": 0,
// Suggestions when no main results:
"hasSuggestions": true,
"suggestionsByCategory": { "Music": [ /* nearby events up to 200km */ ] },
"totalSuggestions": 4,
"suggestionRadius": 200
}
}
```

08 Category-Based Events

Returns events filtered by category. Automatically applies the user's saved location to return nearby results first. Falls back to country-level filtering if coordinates are unavailable.

Method	Endpoint	Auth	Description
GET	/category-events	—	Events filtered by category + user location. Returns suggestions if none found nearby.

Query Parameters

Field	Type	Req.	Description
category	string	No	Category name (URL-encoded). E.g. Music, Sports, Comedy, Tech.
userId	string	No	User ID for location personalisation.

Category names must be URL-encoded. E.g. 'Music & Dance' becomes 'Music%20%26%20Dance'. The server normalises & and whitespace for matching.

09 Category-Based Popular Events

Returns the most popular events in a category, scored by bookings, likes, and tickets sold. Requires a banner or portrait image. Use for featured/trending carousels on the home screen.

Method	Endpoint	Auth	Description
GET	/category-popular-events	—	Top events by popularity score within a category. Requires event image.

Query Parameters

Field	Type	Req.	Description
category	string	No	Category filter (URL-encoded).
searchQuery	string	No	Additional keyword filter applied after category.
limit	number	No	Max results. Min 1, Max 50. Default: 10.

Popularity Score Formula

Score = (totalBookings × 3) + (like × 2) + (totalTicketsSold × 1) — Higher score = more popular.

Response

JSON Response


```
{
"data": {
"category": "Music",
"count": 5,
"events": [
{ "_id": "ticket_abc", "event_name": "WIE Fest", "popularity_score": 568, ... }
]
}
}
```


10 Popular Events (All Categories)

Top events across all categories by popularity score. Same scoring formula as category-based popular events. Use for a global 'Trending Now' section.

Method	Endpoint	Auth	Description
GET	/popular-events	—	Top N events across all categories by popularity score.

Query Parameters

Field	Type	Req.	Description
limit	number	No	Max results. Min 1, Max 50. Default: 10.

11 Filtered Events (Advanced Search)

The most powerful search endpoint. Combines text search, category, subcategory, location, language, date range, and booking date filters in a single call.

Method	Endpoint	Auth	Description
GET	/filtered-events	—	Advanced multi-filter search. Combine any set of filters.

Query Parameters

Field	Type	Req.	Description
category	string	No	Category filter (URL-encoded).
subcategory	string	No	Subcategory exact match.
location	string	No	City / address text (geocoded, then strict match).
latitude	number	No	GPS latitude. Overrides location string.
longitude	number	No	GPS longitude. Overrides location string.
radius	number	No	Radius in km (GPS mode). Default: 200.
searchQuery	string	No	Keyword search on name, category, subcategory.
locationType	string	No	online | offline | hybrid
eventLanguage	string	No	Language name (e.g. English, Hindi).
startDate	string	No	Event start date filter (ISO 8601).
endDate	string	No	Event end date filter (ISO 8601).
bookingStartDate	string	No	Booking window start date filter.
bookingEndDate	string	No	Booking window end date filter.
userId	string	No	User ID for saved location fallback.

Response

JSON Response


```
{
"data": {
"locationAvailable": true,
"locationSource": "gps",
"searchRadius": 200,
"appliedFilters": {
"category": "Music", "subcategory": null,
"locationType": "offline", "eventLanguage": "English",
"startDate": "2025-12-01", "endDate": "2025-12-31"
},
"categories": ["Music"],
"eventsByCategory": { "Music": [ /* events */ ] },
"totalEvents": 3
}
}
```

12 Get Single Event

Fetches full details for a single event by its ticket ID. Tries gRPC first, falls back to the live events list. Returns parent event metadata for sub-events.

Method	Endpoint	Auth	Description
GET	/event/:ticketId	—	Full event details. Works for both main events and sub-events.

Response

JSON Response

```
{
"success": true,
"data": {
"event": { /* full event object */ },
"isSubEvent": false,
"parentEvent": null // { _id, event_name } if isSubEvent=true
}
}
```

13 Event Groups

Groups are organisers / event hosts. Fetch all active groups or a single group by ID.

Method	Endpoint	Auth	Description
GET	/get-active-groups	—	All active event organiser groups.
GET	/group/:groupId	—	Single group by ID.

14 User Location (Save & Get)

Save and retrieve the user's preferred location for event discovery. Called when the user manually sets a city or after GPS permission is granted.

Method	Endpoint	Auth	Description
POST	/user-location	—	Save user's location (city name + optional GPS coordinates).
GET	/user-location	—	Get the user's saved location.

POST /user-location — Request Body

Field	Type	Req.	Description
userId	string	Yes	User ID to associate the location with.
displayName	string	No	Human-readable location name (e.g. 'Mumbai, India').
latitude	number	No	GPS latitude. Stored with the location.
longitude	number	No	GPS longitude. Stored with the location.

Field	Type	Req.	Description
source	string	No	gps | manual. Default: manual.

GET /user-location — Query Params

Field	Type	Req.	Description
userId	string	Yes	User ID to fetch saved location for.

15 Cancelled & Rehosted Events

Method	Endpoint	Auth	Description
GET	/cancelled-events	—	All admin-cancelled events. Optional ?userId for org-specific filter.
GET	/rehosted-events	—	All rehosted (rescheduled) events. Optional ?userId filter.

PART B — Booking Service (Base URL: http://localhost:5007/api/bookings)

16 Register Free Event

Registers the authenticated user for a free event instantly. No payment is required. A QR code is generated and the booking is immediately confirmed.

Method	Endpoint	Auth	Description
POST	/register-free	■	Register for a free event. Immediate confirmation + QR code.

Request Body — application/json

Field	Type	Req.	Description
ticketId	string	Yes	Event ticket ID.
ticketTypeId	string	No	Specific ticket type ID. Defaults to 'Free Entry'.
quantity	number	Yes	Number of tickets. Must be 1 unless restrict_booking=true.

Response (HTTP 201)

JSON Response


```
{
"success": true,
"message": "Free event registration successful",
"data": {
"booking": {
"id": "uuid",
"bookingId": "FR1234567890ABC",
"bookingStatus": "CONFIRMED",
"paymentStatus": "COMPLETED",
"totalAmount": 0,
"qrCode": "..."
},
"qrCode": "..."
}
}
```


17 Create Paid Booking

Initiates a paid booking for a ticketed event. Creates a Razorpay order that the client uses to open the payment sheet. After the user completes payment, call verifyPayment to confirm the booking.

Method	Endpoint	Auth	Description
POST	/create	■	Create booking + Razorpay order. Returns order ID for payment sheet.

Request Body — application/json

Field	Type	Req.	Description
ticketId	string	Yes	Event ticket ID.
ticketTypeId	string	Yes	Ticket type _id (from ticket_types[]).
quantity	number	Yes	Number of tickets. Max 50 if restrict_booking=true, else 1.

Response (HTTP 201)

JSON Response

```
{
"success": true,
"data": {
"booking": {
"id": "uuid",
"bookingId": "BK1234567890ABC",
"subtotal": 1000, // ticket price × qty (GST inclusive)
"platformFee": 5, // WIE flat convenience fee
"totalAmount": 1005, // subtotal + platformFee
"currency": "INR",
"feeBreakdown": {
"lines": [
{ "label": "Ticket price", "amount": 1000 },
{ "label": "Convenience fee", "amount": 5 }
],
"total": 1005
}
},
"razorpayOrder": { "id": "order_abc", "amount": 100500, "currency": "INR" },
"razorpayKeyId": "rzp_test_..."
}
}
```

Amount in razorpayOrder is in PAISE (× 100). Pass razorpayOrder.id, razorpayOrder.amount, and razorpayKeyId directly to the Razorpay React Native SDK.

18 Verify Payment (Razorpay)

Verifies the Razorpay payment signature after the user completes checkout. On success: confirms the booking, generates QR code, creates escrow record, and sends a push notification.

Method	Endpoint	Auth	Description
POST	/verify-payment	■	Verify Razorpay signature → confirm booking → generate QR → create escrow.

Request Body — application/json

Field	Type	Req.	Description
razorpayOrderId	string	Yes	Order ID from razorpayOrder.id returned by /create.
razorpayPaymentId	string	Yes	Payment ID from Razorpay SDK success callback.
razorpaySignature	string	Yes	Signature from Razorpay SDK success callback.

React Native — Razorpay Flow

TypeScript

```
import RazorpayCheckout from 'react-native-razorpay';

// 1. Create booking
const { data } = await bookingApi.post('/create', { ticketId, ticketTypeId, quantity });
const { razorpayOrder, razorpayKeyId, booking } = data.data;

// 2. Open Razorpay payment sheet
const options = {
key: razorpayKeyId,
amount: razorpayOrder.amount, // in paise
currency: razorpayOrder.currency,
order_id: razorpayOrder.id,
name: 'WIE Events',
description: eventName,
prefill: { name: user.name, email: user.email, contact: user.phone },
};
const payment = await RazorpayCheckout.open(options);

// 3. Verify payment
await bookingApi.post('/verify-payment', {
razorpayOrderId: razorpayOrder.id,
razorpayPaymentId: payment.razorpay_payment_id,
razorpaySignature: payment.razorpay_signature,
});
```

19 Create Seated Booking

For events with a seat map. The client sends an array of selected seat IDs. The server validates availability and calculates the total from individual seat prices.

Method	Endpoint	Auth	Description
POST	/create-seated	■	Book specific seats from a venue seat map. Validates availability per seat.

Request Body — application/json

Field	Type	Req.	Description
ticketId	string	Yes	Event ticket ID (must have seating_layout).
selectedSeats	string[]	Yes	Array of seat IDs from the event's seating_layout.seats[].seatId.

Response

JSON Response

```
{
"data": {
"booking": {
"subtotal": 1500, // sum of seat prices
"platformFee": 10, // ■5 × 2 seats
"totalAmount": 1510,
"seatDetails": [
{ "seatId": "A1", "row": "A", "column": 1, "ticketType": "VIP", "price": 750 }
]
},
"razorpayOrder": { "id": "order_xyz", "amount": 151000 }
}
}
```

20 Get Booked Seats

Returns an array of seat IDs that are already booked (CONFIRMED or PENDING). Use this to show unavailable seats on the seat map before the user selects.

Method	Endpoint	Auth	Description
GET	/booked-seats/:ticketId	■	Returns array of booked seat IDs for a venue event.

Response

JSON Response

```
{ "success": true, "data": { "bookedSeats": ["A1", "A2", "B5"] } }
```

21 Check User Booking

Quick check to see if the current user has an active booking for a specific event. Use this to show Register / Already Registered state on the event detail screen.

Method	Endpoint	Auth	Description
GET	/check-booking/:ticketId	■	Returns hasBooked boolean + booking object if exists.

Response

JSON Response

```
{ "success": true, "hasBooked": true, "booking": { /* Booking object */ } }
// or
{ "success": true, "hasBooked": false, "booking": null }
```

22 My Bookings

Returns all bookings for the authenticated user with optional status filter and pagination. Each booking includes a qrPayload object ready for the QR code display screen.

Method	Endpoint	Auth	Description
GET	/my-bookings	■	All user bookings. Optional status filter. Includes qrPayload per booking.

Query Parameters

Field	Type	Req.	Description
status	string	No	Filter: CONFIRMED | PENDING | CANCELLED
limit	number	No	Results per page. Default: 50.
skip	number	No	Number to skip (offset pagination). Default: 0.

Booking Status Values

Status	Meaning
PENDING	Booking created, awaiting payment verification.
CONFIRMED	Payment verified or free event registered. QR code generated.
CANCELLED	Cancelled by user or admin. Refund status reflects refund state.

23 Get Booking by ID

Returns full details of a single booking including the QR payload, event link, and event verification code. Use for the booking detail / ticket display screen.

Method	Endpoint	Auth	Description
GET	/:bookingId	■	Full booking details + qrPayload + event_link + event_code.

Response

JSON Response


```
{
"data": {
"booking": {
"id": "uuid",
"bookingId": "BK1234567890ABC",
"bookingStatus": "CONFIRMED",
"paymentStatus": "COMPLETED",
"totalAmount": 1005,
"qrCode": "...", // PNG QR code
"event_link": "https://meet.google.com/...", // for online events
"event_code": "VERIFY123",
"qrPayload": {
"bookingId": "BK123",
"eventName": "WIE Music Fest",
"ticketType": "VIP",
"quantity": 2,
"holderName": "Jane Doe",
"eventDate": "2025-12-20",
"venue": "BKC Arena",
"totalAmount": 1005
}
}
}
}
```


24 Cancel Booking

Cancels a confirmed booking. Enforces a cancellation window — must be at least 4 hours before the event. Initiates a Razorpay refund for paid bookings following the refund policy.

Method	Endpoint	Auth	Description
POST	/:bookingId/cancel	■	Cancel booking. Enforces 4-hour window. Initiates refund for paid bookings.

Request Body — application/json

Field	Type	Req.	Description
cancellationReason	string	No	Reason for cancellation. Stored for audit trail.

Cancellation Rules

Timing	Result
4+ hours before	Cancellation allowed. Refund calculated by policy.
< 4 hours before	Cancellation blocked — event too close.
After event starts	Cancellation blocked — event has passed.

Refund Policy (DEFAULT)

Window	Refund
48+ hours before	100% refund of ticket value + platform fee.
24–48 hours	75% refund of ticket value only (no platform fee refund).
4–24 hours	50% refund of ticket value only.
Free event	No refund — registration simply cancelled.

Response

JSON Response

```
{
"success": true,
"message": "Booking cancelled successfully",
"data": {
"booking": { /* updated booking */ },
"refundStatus": "PROCESSING", // COMPLETED | PROCESSING | NOT_APPLICABLE
"cancellationStats": { "totalCancellations": 1, "totalCancelledTickets": 2 }
}
}
```

25 Track Refund

Method	Endpoint	Auth	Description
GET	/:bookingId/refund/track	■	Get real-time refund status and refund transaction history.

Response

JSON Response

```
{
"data": {
"refundSummary": {
"status": "COMPLETED",
"isCompleted": true,
"refundAmount": 1000,
"refundId": "rfnd_abc",
"refundInitiatedAt": "2025-12-01T12:00:00Z",
"refundProcessedAt": "2025-12-01T12:05:00Z",
"statusLabel": "Refund Successful",
"statusColor": "green" // green | red | yellow | gray
}
}
}
```

26 Cancelled & Rehosted Bookings

Fetches the user's cancelled bookings (including admin-cancelled events) and relevant rehosted events. Auto-fixes stale bookings where the event was cancelled by the host but the booking row was not yet updated.

Method	Endpoint	Auth	Description
GET	/my-cancelled-bookings	■	User's cancelled bookings. Auto-fixes stale admin-cancelled entries.
GET	/my-rehosted-bookings	■	Rehosted events relevant to user's cancelled bookings.

my-cancelled-bookings cross-references the global cancelled events list (gRPC) with the user's bookings. It automatically marks stale CONFIRMED bookings as CANCELLED if the host cancelled the event, saving a manual support step.

27 Unread Count & Mark Read

Track which booking notifications the user has seen. Use unread-count for badge counts on the bookings tab. Call mark-read when the user opens the bookings list.

Method	Endpoint	Auth	Description
GET	/unread-count	■	Count unread bookings by status (confirmed, pending, cancelled).
POST	/mark-read	■	Mark bookings as read by ID array, status filter, or both.

GET /unread-count — Response

JSON Response

```
{ "data": { "confirmed": 2, "pending": 0, "cancelled": 1 } }
```

POST /mark-read — Request Body

Field	Type	Req.	Description
bookingIds	string[]	No	Array of booking IDs to mark read.
statuses	string[]	No	Mark all bookings with these statuses as read (e.g. ['CONFIRMED']).

28 Booking Statistics (Organiser)

Analytics endpoints for event organisers. No authentication required — intended for dashboard displays. Returns daily, monthly, and ticket-type breakdowns.

Method	Endpoint	Auth	Description
GET	/stats/daily/:ticketId	—	Daily booking counts + revenue for an event.
GET	/stats/monthly/:ticketId	—	Monthly stats. Optional ?startDate and ?endDate filters.
GET	/stats/ticket-types/:ticketId	—	Sales breakdown per ticket type.

PART C — Interaction Service (Base URL: http://localhost:5007/api/interactions)

29 Like / Unlike Event

Toggles like on an event. Calling again when already liked removes the like. The like count on the ticket is updated via gRPC. Both POST and DELETE routes point to the same toggle handler.

Method	Endpoint	Auth	Description
POST	/:ticketId/like	■	Like (if not liked) or unlike (if already liked). Toggle.
DELETE	/:ticketId/like	■	Explicit unlike alias. Same controller as POST.

Response

JSON Response

```
// On like: { "success": true, "liked": true, "message": "Event liked" }
// On unlike: { "success": true, "liked": false, "message": "Event unliked" }
```

30 Save / Unsave Event

Toggles save (bookmark) on an event. Same toggle behaviour as like.

Method	Endpoint	Auth	Description
POST	/:ticketId/save	■	Save (if not saved) or unsave (if already saved). Toggle.
DELETE	/:ticketId/save	■	Explicit unsave alias. Same controller as POST.

JSON Response

```
// On save: { "success": true, "saved": true }
// On unsave: { "success": true, "saved": false }
```

31 Share Event

Records a share event interaction. Increments the event's share count via gRPC. Use this whenever the user shares the event via any platform.

Method	Endpoint	Auth	Description
POST	/:ticketId/share	■	Record a share. Increments share count. Optional platform metadata.

Request Body — application/json

Field	Type	Req.	Description
shareMethod	string	No	How it was shared: link | whatsapp | copy | etc.
platform	string	No	Platform used: WhatsApp | Instagram | Twitter | etc.

JSON Response

```
{ "success": true, "message": "Event shared successfully", "data": { "shareCount": 42 } }
```

32 Record View

Records a view event. Deduped with a 24-hour window per user per event — only one view per user per day is counted. Call this when the event detail screen becomes visible.

Method	Endpoint	Auth	Description
POST	/:ticketId/view	■	Record event view. Deduped per user per 24 hours.

JSON Response

```
{ "success": true, "message": "View recorded" }
```

33 Get Event Stats

Returns aggregated interaction stats for an event plus the current user's specific interaction states (liked, saved, shared). Auth is optional — if no token, userInteractions fields are all false.

Method	Endpoint	Auth	Description
GET	/:ticketId/stats	—	Event stats: likes, views, shares, saves + current user's interaction state.

Response

JSON Response

```
{
"data": {
"stats": {
"like": 38, "likes": 38,
"share": 12, "shares": 12,
"views": 540, "view": 540,
"saves": 25, "save": 25,
"totalBookings": 142,
"totalRevenue": 71000,
"totalTicketsSold": 142
},
"userInteractions": {
"liked": true,
"saved": false,
"shared": true
}
}
}
```

34 Submit Feedback

Allows users to rate and review an event. Requires a confirmed booking for the event. One feedback per user per event — calling again updates the existing submission.

Method	Endpoint	Auth	Description
POST	/:ticketId/feedback	■	Submit/update rating and comment. Must have CONFIRMED booking.

Request Body — application/json

Field	Type	Req.	Description
rating	number	Yes	Rating from 1 to 5.
comment	string	No	Optional review text.

35 User Liked & Saved Events

Returns lists of ticket IDs that the user has liked or saved. Use these to restore UI state (filled heart / bookmark icon) on event cards across the app.

Method	Endpoint	Auth	Description
GET	/liked-events	■	List of ticketIds the user has liked.
GET	/saved-events	■	List of ticketIds the user has saved.

Query Parameters (both endpoints)

Field	Type	Req.	Description
limit	number	No	Max results. Default: 50.
skip	number	No	Offset for pagination. Default: 0.

JSON Response

```
{ "success": true, "data": { "ticketIds": ["ticket_abc", "ticket_xyz"], "total": 2 } }
```

36 Full Route Reference

Event Listing — http://localhost:5005/api/tickets (no auth required)

Method	Endpoint	Auth	Description
GET	/live-events	—	All live events
GET	/all-events	—	All events with distance
GET	/nearby-events	—	Events within radius
GET	/initial-events	—	Home screen personalised events
GET	/search-by-name	—	Search by keyword
GET	/search-by-location	—	Search by GPS or city
GET	/category-events	—	Events by category
GET	/category-popular-events	—	Popular events by category
GET	/popular-events	—	Top popular events all categories
GET	/filtered-events	—	Advanced multi-filter search
GET	/event/:ticketId	—	Single event details
GET	/get-active-groups	—	All organiser groups
GET	/group/:groupId	—	Single group details
POST	/user-location	—	Save user location
GET	/user-location	—	Get saved user location
GET	/cancelled-events	—	All cancelled events
GET	/rehosted-events	—	All rehosted events

Booking Service — http://localhost:5007/api/bookings (auth required)

Method	Endpoint	Auth	Description
POST	/register-free	■	Register for free event
POST	/create	■	Create paid booking + Razorpay order
POST	/verify-payment	■	Verify Razorpay signature → confirm
POST	/create-seated	■	Book specific seats
GET	/booked-seats/:ticketId	■	Booked seat IDs for venue event
GET	/check-booking/:ticketId	■	Has user booked this event?
GET	/my-bookings	■	All user bookings
GET	/my-cancelled-bookings	■	Cancelled bookings (auto-fixes stale)
GET	/my-rehosted-bookings	■	Rehosted events for user's cancellations
GET	/cancellation-stats	■	User's cancellation stats

Method	Endpoint	Auth	Description
GET	/unread-count	■	Unread booking counts by status
POST	/mark-read	■	Mark bookings as read
GET	/:bookingId	■	Single booking + QR payload
POST	/:bookingId/cancel	■	Cancel booking + initiate refund
GET	/:bookingId/refund/track	■	Track refund status
GET	/stats/daily/:ticketId	—	Daily booking stats
GET	/stats/monthly/:ticketId	—	Monthly booking stats
GET	/stats/ticket-types/:tid	—	Sales by ticket type

Interaction Service — http://localhost:5007/api/interactions

Method	Endpoint	Auth	Description
POST	/:ticketId/like	■	Toggle like/unlike
DELETE	/:ticketId/like	■	Explicit unlike alias
POST	/:ticketId/save	■	Toggle save/unsave
DELETE	/:ticketId/save	■	Explicit unsave alias
POST	/:ticketId/share	■	Record share + increment count
POST	/:ticketId/view	■	Record view (24h dedupe)
GET	/:ticketId/stats	—	Event stats + user interaction state
POST	/:ticketId/feedback	■	Submit/update rating
GET	/liked-events	■	User's liked ticketIds
GET	/saved-events	■	User's saved ticketIds

37 React Native Integration Examples

Axios Instances — Three Services

TypeScript

```typescript
// src/services/ticketingService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';
const TICKET_BASE = (Config.API_BASE_URL || 'http://10.0.2.2:5005') + '/api/tickets';
const BOOKING_BASE = (Config.TRANSACTION_API_URL || 'http://10.0.2.2:5007') + '/api/bookings';
const INTERACT_BASE = (Config.TRANSACTION_API_URL|| 'http://10.0.2.2:5007') + '/api/interactions';
export const ticketApi = axios.create({ baseURL: TICKET_BASE, timeout: 20000 });
export const bookingApi = axios.create({ baseURL: BOOKING_BASE, timeout: 30000 });
export const interactApi = axios.create({ baseURL: INTERACT_BASE, timeout: 15000 });
// Inject token on authenticated apis
[bookingApi, interactApi].forEach(api => {
api.interceptors.request.use(async cfg => {
const t = await AsyncStorage.getItem('token');
if (t) cfg.headers.Authorization = `Bearer ${t}`;
return cfg;
});
});
// ■■ Event Listing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const getInitialEvents = (userId: string) =>
ticketApi.get('/initial-events', { params: { userId } }).then(r => r.data);
export const searchByName = (q: string, userId?: string) =>
ticketApi.get('/search-by-name', { params: { searchQuery: q, userId } }).then(r => r.data);
export const getFilteredEvents = (params: Record) =>
ticketApi.get('/filtered-events', { params }).then(r => r.data);
export const getEventById = (id: string) =>
ticketApi.get(`/event/${id}`).then(r => r.data.data);
export const getNearbyEvents = (lat: number, lng: number, radius = 30) =>
ticketApi.get('/nearby-events', { params: { latitude: lat, longitude: lng, radius } }).then(r => r.data);
// ■■ Booking ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const registerFree = (ticketId: string, quantity = 1) =>
bookingApi.post('/register-free', { ticketId, quantity }).then(r => r.data);
export const createBooking = (ticketId: string, ticketTypeId: string, quantity: number) =>
bookingApi.post('/create', { ticketId, ticketTypeId, quantity }).then(r => r.data);
export const verifyPayment = (orderId: string, paymentId: string, sig: string) =>
bookingApi.post('/verify-payment', { razorpayOrderId: orderId, razorpayPaymentId: paymentId, razorpaySignature: sig
}).then(r => r.data);
export const getMyBookings = (status?: string) =>
bookingApi.get('/my-bookings', { params: { status } }).then(r => r.data);
export const cancelBooking = (bookingId: string, reason?: string) =>
bookingApi.post(`/${bookingId}/cancel`, { cancellationReason: reason }).then(r => r.data);
// ■■ Interactions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const toggleLike = (ticketId: string) =>
interactApi.post(`/${ticketId}/like`).then(r => r.data);
export const toggleSave = (ticketId: string) =>
interactApi.post(`/${ticketId}/save`).then(r => r.data);
export const getEventStats = (ticketId: string) =>
interactApi.get(`/${ticketId}/stats`).then(r => r.data.data);
```

```
export const recordView = (ticketId: string) =>
interactApi.post(`/${ticketId}/view`).catch(() => {}); // silent fail
```

Event Detail Screen — Full Load Pattern

TypeScript

```
// When user opens an event detail screen:
const loadEvent = async (ticketId: string) => {
const [event, stats] = await Promise.all([
getEventById(ticketId),
getEventStats(ticketId),
]);
setEvent(event.event);
setLiked(stats.userInteractions.liked);
setSaved(stats.userInteractions.saved);
setLikeCount(stats.stats.like);
setViewCount(stats.stats.views);
// Record view silently
recordView(ticketId);
// Check if user already booked
const { hasBooked, booking } =
await bookingApi.get(`/check-booking/${ticketId}`).then(r => r.data);
setHasBooked(hasBooked);
setMyBooking(booking);
};
```

Full Razorpay Booking Flow

TypeScript

```
import RazorpayCheckout from 'react-native-razorpay';
const bookTicket = async (ticketId: string, typeId: string, qty: number) => {
setLoading(true);
try {
// Step 1: Create booking
const { data: { booking, razorpayOrder, razorpayKeyId } } =
await bookingApi.post('/create', { ticketId, ticketTypeId: typeId, quantity: qty });
// Step 2: Open payment sheet
const payment = await RazorpayCheckout.open({
key: razorpayKeyId,
amount: razorpayOrder.amount, // paise
currency: 'INR',
order_id: razorpayOrder.id,
name: 'WIE Events',
prefill: { name: user.name, email: user.email, contact: user.phone },
});
// Step 3: Verify on backend
const verified = await verifyPayment(
razorpayOrder.id,
payment.razorpay_payment_id,
payment.razorpay_signature
);
showToast('Booking Confirmed! Check your QR code in My Bookings.');
navigation.navigate('BookingDetail', { bookingId: booking.id });
} catch (err: any) {
```

```
if (err.code === 'PAYMENT_CANCELLED') {
showToast('Payment cancelled.');
} else {
Alert.alert('Booking Failed', err.message);
}
} finally {
setLoading(false);
}
};
```

Environment Variables (.env)

.env

```
# Event listing (User Service)
API_BASE_URL=http://localhost:5005
# Booking + Interactions (Transaction Service)
TRANSACTION_API_URL=http://localhost:5007
# For Android emulator, replace localhost with 10.0.2.2
# API_BASE_URL=http://10.0.2.2:5005
# TRANSACTION_API_URL=http://10.0.2.2:5007
```

All booking amounts are in Indian Rupees (INR). Razorpay amounts are in paise (amount × 100). Platform fee is a flat ■5 per booking (WIE convenience fee). GST is the event organiser's responsibility and is already included in the ticket_price if applicable.